LAPORAN PRAKTIKUM

Algoritma dan Struktur Data

Analisis Kompleksitas Algoritma

DISUSUN OLEH:

FAQROEL RAZI

2025573010033

TEKNIK INFORMATIKA

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan laporan praktikum mata kuliah Praktikum Algoritma dan Struktur Data ini. Laporan ini disusun sebagai dokumentasi dari kegiatan praktikum yang telah dilaksanakan, khususnya dalam mempelajari **Analisis Kompleksitas Algoritma**.

Dalam praktikum ini, saya mempelajari konsep dasar analisis kompleksitas algoritma, meliputi pengukuran efisiensi waktu eksekusi (*time complexity*) dan penggunaan memori (*space complexity*) pada suatu program. Selain itu, saya juga mempelajari notasi Big O sebagai metode untuk menggambarkan performa algoritma berdasarkan pertumbuhan data masukan. Kegiatan ini membantu saya memahami pentingnya memilih algoritma yang efisien agar program dapat berjalan lebih optimal, terutama saat menangani data dalam jumlah besar.

Saya mengucapkan terima kasih kepada Bapak Muhammad Reza Zulman, M.Sc., selaku dosen pengampu, atas bimbingan dan arahan yang telah diberikan selama praktikum berlangsung. Semoga laporan ini dapat menjadi refleksi dari proses pembelajaran yang telah saya jalani dalam memahami analisis kompleksitas algoritma.

Lhokseumawe, 24 April 2026

Penulis

DAFTAR ISI

JUDUL ..... **i**
KATA PENGANTAR ..... **ii**
DAFTAR ISI ..... **iii**
BAB I PENDAHULUAN **1**
1.1. Latar Belakang 1
1.2. Identifikasi Masalah 1
1.3. Rumusan Masalah 2
BAB II LANDASAN TEORI **3**
2.1. Kompleksitas Algoritma 3
2.2. Big O 3
2.3. Space Complexity 3
2.4. Refactor / Pola Kompleksitas 4
BAB III METODOLOGI PRAKTIKUM **5**
3.1. Alat dan Bahan 5
3.2. Prosedur Praktikum 5
3.2.1. Setup Project 5
3.2.2. Project Analisis dan Refactor 6
3.2.3. Project Visualisasi Pertumbuhan Big O 9
BAB IV HASIL DAN PEMBAHASAN **13**
4.1. Hasil Praktikum 13
4.2. Pembahasan 13
BAB V PENUTUP **14**
5.1. Kesimpulan 14
5.2. Saran 14
DAFTAR PUSTAKA **15**

BAB I`

PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi informasi menuntut perangkat lunak yang mampu bekerja secara cepat, efisien, dan stabil. Dalam pengembangan program, tidak cukup hanya membuat kode yang dapat berjalan, tetapi juga perlu memperhatikan seberapa optimal algoritma yang digunakan. Algoritma yang kurang efisien dapat menyebabkan waktu proses lebih lama dan penggunaan memori yang berlebihan, terutama ketika data yang diolah berjumlah besar.

Analisis kompleksitas algoritma merupakan metode untuk mengukur performa suatu algoritma berdasarkan kebutuhan waktu eksekusi dan penggunaan ruang memori. Dengan memahami kompleksitas, programmer dapat membandingkan beberapa solusi untuk suatu masalah dan memilih pendekatan terbaik. Konsep ini menjadi dasar penting dalam ilmu komputer karena berhubungan langsung dengan kualitas perangkat lunak yang dibangun.

Pada praktikum ini, analisis kompleksitas algoritma dipelajari menggunakan bahasa pemrograman JavaScript yang dijalankan melalui Node.js. Pemanfaatan JavaScript memberikan kemudahan dalam pengujian algoritma secara langsung melalui terminal, sehingga mahasiswa dapat memahami hubungan antara teori kompleksitas dengan implementasi program nyata.

1.2. Identifikasi Masalah

Berdasarkan latar belakang di atas, maka dapat diidentifikasi beberapa masalah sebagai berikut:

1. Masih kurangnya pemahaman mengenai pentingnya efisiensi algoritma dalam pemrograman.

2. Sulitnya membedakan algoritma yang memiliki performa baik dan kurang baik.
3. Kurangnya pemahaman tentang notasi Big O sebagai alat ukur kompleksitas.
4. Belum dipahaminya hubungan antara struktur kode dengan penggunaan memori program.
5. Perlunya latihan implementasi analisis kompleksitas menggunakan JavaScript.

1.3. Rumusan Masalah

Adapun rumusan masalah dalam praktikum ini adalah sebagai berikut:

1. Apa yang dimaksud dengan kompleksitas algoritma?
2. Bagaimana cara kerja notasi Big O dalam mengukur efisiensi algoritma?
3. Apa yang dimaksud dengan *space complexity* pada program?
4. Bagaimana refactor kode dapat memengaruhi kompleksitas algoritma?
5. Bagaimana implementasi analisis kompleksitas algoritma menggunakan JavaScript dan Node.js?

BAB II

LANDASAN TEORI

2.1. Kompleksitas Algoritma

Kompleksitas algoritma adalah ukuran yang digunakan untuk mengetahui tingkat efisiensi suatu algoritma dalam menyelesaikan masalah. Pengukuran ini biasanya dilihat dari dua aspek utama, yaitu waktu eksekusi dan penggunaan memori. Semakin kecil sumber daya yang dibutuhkan, maka algoritma dianggap semakin efisien.

Kompleksitas algoritma penting karena program sering kali harus menangani data dalam jumlah besar. Algoritma sederhana mungkin bekerja baik pada data kecil, tetapi menjadi lambat saat skala data meningkat. Oleh karena itu, analisis kompleksitas membantu pengembang memprediksi performa program sebelum digunakan secara nyata.

2.2. Big O

Big O Notation adalah cara untuk menggambarkan laju pertumbuhan waktu eksekusi atau penggunaan memori suatu algoritma terhadap ukuran input. Notasi ini fokus pada perilaku algoritma saat data semakin besar, bukan pada waktu nyata dalam detik.

Contoh Big O yang umum digunakan:

- **O(1)** → Waktu konstan, tidak dipengaruhi jumlah data.
- **O(log n)** → Pertumbuhan lambat, contoh pencarian biner.
- **O(n)** → Linear, sebanding dengan jumlah data.
- **O(n log n)** → Umum pada algoritma sorting efisien.
- **O(n^2)** → Kuadratik, sering terjadi pada nested loop.
- **O(2^n)** → Eksponensial, sangat lambat untuk data besar.

2.3. Space Complexity

Space complexity adalah ukuran jumlah memori yang dibutuhkan algoritma selama proses berjalan. Selain waktu eksekusi, penggunaan memori juga menjadi faktor penting dalam efisiensi program.

Sebagai contoh, algoritma yang membuat banyak array tambahan akan menggunakan memori lebih besar dibanding algoritma yang memproses data secara langsung. Dalam sistem dengan sumber daya terbatas, optimasi memori sangat diperlukan agar program tetap stabil.

2.4. Refactor / Pola Kompleksitas

Beberapa pola kompleksitas yang umum dijumpai dalam algoritma antara lain:

1. **Konstan (O(1))**
   Waktu eksekusi tetap meskipun ukuran input bertambah.
2. **Linear (O(n))**
   Waktu bertambah sebanding dengan jumlah data.
3. **Kuadratik ($O(n^2)$)**
   Biasanya terjadi pada perulangan bersarang.
4. **Logaritmik (O(log n))**
   Terjadi pada algoritma pencarian efisien seperti binary search.
5. **Eksponensial ($O(2^n)$)**
   Pertumbuhan sangat cepat dan tidak efisien untuk data besar.

BAB III
METODOLOGI PRAKTIKUM

3.1. Alat dan Bahan

Berdasarkan praktikum yang dilakukan, alat dan bahan yang diperlukan dalam implementasi javascript dasar adalah:

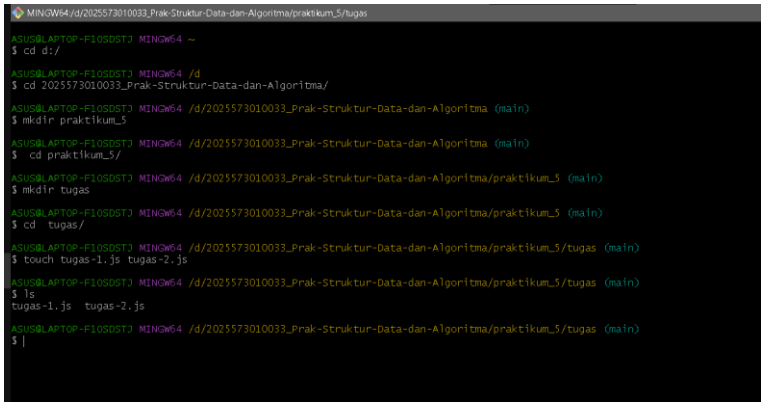
- Komputer dan Laptop
- IDE Visual Studio Code
- Node.js
- Git
- GitHub

3.2. Prosedur Praktikum

3.2.1. Setup Project

Dalam praktikum ini, terdapat dua project yaitu project "Analisis dan Refactor" dan "Visualisasi Pertumbuhan Big O", setup yang diperlukan adalah:

1. Buka git bash dan navigasikan ke folder repository GitHub yang telah di clone.
2. Buat direktori praktikum-5
3. Masuk ke direktori praktikum-5
4. Buat direktori tugas
5. Masuk ke direktori tugas
6. Buat file tugas-1.js untuk project Analisis dan Refactor
7. Buat file tugas-2.js untuk project Visualisasi Pertumbuhan Big O



3.2.2. Project Analisis dan Refactor

Berdasarkan tugas praktikum yang diberikan, project Analisis Kompleksitas Algoritma ini diminta untuk membuat tiga fungsi menggunakan JavaScript di Node.js. Fungsi A mencari irisan dua array dengan versi $O(n^2)$ dan $O(n)$, Fungsi B mengelompokkan kata anagram, dan Fungsi C mengecek apakah ada dua angka yang jumlahnya sama dengan kuadrat angka lain menggunakan versi $O(n^3)$ dan $O(n \log n)$. Selain itu, setiap fungsi diminta menuliskan analisis Big O serta mengukur waktu eksekusi program.

3.2.2.1. Membuat syntax

Dengan memenuhi permintaan praktikum, berikut syntax JavaScript:

```javascript
// FUNGSI A: ARRAY INTERSECTION - O(n²) Brute Force
function intersectionBruteForce(arr1, arr2) {
  const result = [];
  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j] && !result.includes(arr1[i])) {
        result.push(arr1[i]);
      }
    }
  }
  return result;
}

// FUNGSI A: ARRAY INTERSECTION - O(n) Optimized dengan Set
function intersectionOptimized(arr1, arr2) {
  const set2 = new Set(arr2);
  const result = [];
  const seen = new Set();

  for (let num of arr1) {
    if (set2.has(num) && !seen.has(num)) {
      result.push(num);
      seen.add(num);
    }
  }
  return result;
}

// FUNGSI B: GROUP ANAGRAMS - O(n * k log k)
function groupAnagrams(words) {
  const map = new Map();

  for (let word of words) {
    const sorted = word.split("").sort().join("");

    if (!map.has(sorted)) {
      map.set(sorted, []);
    }
    map.get(sorted).push(word);
  }
```

```
  return Array.from(map.values());
}

// FUNGSI C: TWO SUM EQUALS THIRD - O(n³) Brute Force
function twoSumEqualsThirdBruteForce(arr) {
  for (let i = 0; i < arr.length; i++) {
    for (let j = 0; j < arr.length; j++) {
      for (let k = 0; k < arr.length; k++) {
        if (i !== j && i !== k && j !== k && arr[i] + arr[j] === arr[k]) {
          return true;
        }
      }
    }
  }
  return false;
}

// FUNGSI C: TWO SUM EQUALS THIRD - O(n log n) Optimized
function twoSumEqualsThirdOptimized(arr) {
  if (arr.length < 3) return false;

  const sorted = [...arr].sort((a, b) => a - b);

  for (let k = 0; k < sorted.length; k++) {
    const target = sorted[k];
    let left = 0;
    let right = sorted.length - 1;

    while (left < right) {
      if (left === k) left++;
      if (right === k) right--;

      if (left >= right) break;

      const sum = sorted[left] + sorted[right];
      if (sum === target) {
        return true;
      } else if (sum < target) {
        left++;
      } else {
        right--;
      }
    }
```

```javascript
  return false;
}

// TESTING

console.log("=".repeat(70));
console.log("TEST FUNGSI A: ARRAY INTERSECTION");
console.log("=".repeat(70));
const arr1A = [1, 2, 3, 4, 5];
const arr2A = [3, 4, 5, 6, 7];

console.log(`Array 1: [${arr1A}]`);
console.log(`Array 2: [${arr2A}]`);
console.log(`Hasil: [${intersectionBruteForce(arr1A, arr2A)}]\n`);

const largeArr1 = Array.from({ length: 10000 }, (_, i) => i);
const largeArr2 = Array.from({ length: 10000 }, (_, i) => i + 5000);

console.time("Brute Force O(n²)");
intersectionBruteForce(largeArr1, largeArr2);
console.timeEnd("Brute Force O(n²)");

console.time("Optimized O(n)");
intersectionOptimized(largeArr1, largeArr2);
console.timeEnd("Optimized O(n)");

console.log("\n" + "=".repeat(70));
console.log("TEST FUNGSI B: GROUP ANAGRAMS");
console.log("=".repeat(70));
const words = ["eat", "tea", "tan", "ate", "nat", "bat"];
console.log(`Input: [${words.map((w) => "'" + w + "'").join(", ")}]`);

const grouped = groupAnagrams(words);
console.log("Output:");
grouped.forEach((group) => {
  console.log(`  [${group.map((w) => "'" + w + "'").join(", ")}]`);
});

const largeWordList = [];
const wordPatterns = ["abc", "bca", "cab", "aaa", "bbb", "xyz", "yzx"];
for (let i = 0; i < 5000; i++) {
  largeWordList.push(wordPatterns[i % wordPatterns.length]);
}
```

```javascript
console.time("Group Anagrams");
groupAnagrams(largeWordList);
console.timeEnd("Group Anagrams");

console.log("\n" + "=".repeat(70));
console.log("TEST FUNGSI C: TWO SUM EQUALS THIRD");
console.log("=".repeat(70));
const arr1C = [1, 2, 3, 5, 6];
console.log(`Array: [${arr1C}]`);
console.log(`Brute Force: ${twoSumEqualsThirdBruteForce(arr1C)}`);
console.log(`Optimized: ${twoSumEqualsThirdOptimized(arr1C)}`);

const arr2C = [1, 2, 5, 7, 10];
console.log(`\nArray: [${arr2C}]`);
console.log(`Brute Force: ${twoSumEqualsThirdBruteForce(arr2C)}`);
console.log(`Optimized: ${twoSumEqualsThirdOptimized(arr2C)}`);

const largeArr = Array.from({ length: 500 }, (_, i) => i + 1);

console.time("Brute Force O(n³)");
twoSumEqualsThirdBruteForce(largeArr);
console.timeEnd("Brute Force O(n³)");

console.time("Optimized O(n log n)");
twoSumEqualsThirdOptimized(largeArr);
console.timeEnd("Optimized O(n log n)");

console.log("\n" + "=".repeat(70));
```

3.2.2.2. Output program

```
ASUS@LAPTOP-F1OSDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma (main)
$ node praktikum-5/tugas/tugas-1.js
======================================================================
TEST FUNGSI A: ARRAY INTERSECTION
======================================================================
Array 1: [1,2,3,4,5]
Array 2: [3,4,5,6,7]
Hasil: [3,4,5]

Brute Force O(n²): 294.019ms
Optimized O(n): 3.295ms

======================================================================
TEST FUNGSI B: GROUP ANAGRAMS
======================================================================
Input: ['eat', 'tea', 'tan', 'ate', 'nat', 'bat']
Output:
  ['eat', 'tea', 'ate']
  ['tan', 'nat']
  ['bat']
Group Anagrams: 4.502ms

======================================================================
TEST FUNGSI C: TWO SUM EQUALS THIRD
======================================================================
Array: [1,2,3,5,6]
Brute Force: true
Optimized: true

Array: [1,2,5,7,10]
Brute Force: true
Optimized: true
Brute Force O(n³): 0.016ms
Optimized O(n log n): 0.11ms

======================================================================

ASUS@LAPTOP-F1OSDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma (main)
$
```

Output program ini sesuai permintaan praktikum, di mana setiap fungsi berhasil menampilkan hasil pengolahan data sesuai tugas, seperti irisan dua array, pengelompokan anagram, serta pengecekan kondisi jumlah dua angka terhadap kuadrat angka lain beserta analisis kompleksitasnya.

3.2.3. Project Visualisasi Pertumbuhan Big O

Berdasarkan tugas praktikum yang diberikan, project Visualisasi Pertumbuhan Big O ini diminta untuk membuat function benchmarkSemua(ukuranData) menggunakan JavaScript di Node.js yang mengukur dan menampilkan waktu eksekusi empat fungsi, yaitu O(1), O(n), O(n log n), dan O(n^2). Program dijalankan dengan beberapa ukuran data seperti 100,

500, 1000, 5000, dan 10000, kemudian hasilnya diamati untuk melihat pola pertumbuhan waktu eksekusi serta membandingkannya dengan teori Big O.

3.2.3.1. Membuat Syntax

Dengan memenuhi permintaan pada praktikum, berikut syntax JavaScript:

```javascript
// TUGAS 2: VISUALISASI PERTUMBUHAN BIG O

// O(1) - Constant time
function fn_O1(n) {
  return n + 1;
}

// O(n) - Linear time
function fn_On(n) {
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += i;
  }
  return sum;
}

// O(n log n) - Linearithmic time
function fn_OnLogn(n) {
  let count = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < Math.log2(n); j++) {
      count++;
    }
  }
  return count;
}

// O(n²) - Quadratic time
function fn_On2(n) {
  let count = 0;
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      count++;
    }
  }
  return count;
}

// Benchmark function
```

```javascript
// Benchmark function
function benchmarkSemua(ukuranData) {
  console.log("=".repeat(80));
  console.log("BENCHMARKING BIG O GROWTH PATTERNS");
  console.log("=".repeat(80));
  console.log("Data Sizes: " + ukuranData.join(", "));
  console.log("=".repeat(80));

  const results = {
    "O(1)": [],
    "O(n)": [],
    "O(n log n)": [],
    "O(n²)": [],
  };

  for (let size of ukuranData) {
    console.log(`\n--- Size: ${size} ---`);

    // Benchmark O(1)
    console.time("O(1)");
    fn_O1(size);
    console.timeEnd("O(1)");

    // Benchmark O(n)
    console.time("O(n)");
    fn_On(size);
    console.timeEnd("O(n)");

    // Benchmark O(n log n)
    console.time("O(n log n)");
    fn_OnLogn(size);
    console.timeEnd("O(n log n)");

    // Benchmark O(n²)
    console.time("O(n²)");
    fn_On2(size);
    console.timeEnd("O(n²)");
  }

  console.log("\n" + "=".repeat(20));
  console.log("ANALISIS PERTUMBUHAN");
  console.log("=".repeat(20));
  console.log(`
Observasi dari hasil benchmark:

1. O(1) - Konstant Time:
   Waktu eksekusi hampir sama untuk semua ukuran data
   Tidak terpengaruh oleh besarnya input

2. O(n) - Linear Time:
   Waktu tumbuh secara linear dengan ukuran data
   Jika n meningkat 5x, waktu juga meningkat ~5x

3. O(n log n) - Linearithmic Time:
   Waktu tumbuh lebih lambat dari O(n²) tapi lebih cepat dari O(n)
   Tumbuh linear dengan penambahan logaritmik

4. O(n²) - Quadratic Time:
   Waktu tumbuh sangat cepat (eksponensial dari n)
   Jika n meningkat 5x, waktu meningkat ~25x
   Paling lambat di antara semua algoritma

Kesimpulan:
Pola pertumbuhan konsisten dengan teori Big O. Semakin tinggi kompleksitas,
semakin signifikan peningkatan waktu eksekusi seiring bertambahnya input size.
  `);
}

// Panggil benchmark dengan ukuran data yang berbeda
benchmarkSemua([100, 500, 1000, 5000, 10000]);
```

3.2.3.2. Output program

```
ASUS@LAPTOP-F1OSDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma (main)
$ node praktikum-5/tugas/tugas-2.js
================================================================================
BENCHMARKING BIG O GROWTH PATTERNS
================================================================================
Data Sizes: 100, 500, 1000, 5000, 10000
================================================================================

--- Size: 100 ---
O(1): 0.029ms
O(n): 0.04ms
O(n log n): 0.109ms
O(n²): 0.474ms

--- Size: 500 ---
O(1): 0.008ms
O(n): 0.016ms
O(n log n): 0.253ms
O(n²): 0.787ms

--- Size: 1000 ---
O(1): 0.065ms
O(n): 0.013ms
O(n log n): 0.576ms
O(n²): 1.806ms

--- Size: 5000 ---
O(1): 0.019ms
O(n): 0.59ms
O(n log n): 0.218ms
O(n²): 23.94ms

--- Size: 10000 ---
O(1): 0.006ms
O(n): 0.074ms
O(n log n): 0.628ms
O(n²): 109.737ms

====================
```

```
====================
ANALISIS PERTUMBUHAN
====================

Observasi dari hasil benchmark:

1. O(1) - Konstant Time:
   Waktu eksekusi hampir sama untuk semua ukuran data
   Tidak terpengaruh oleh besarnya input

2. O(n) - Linear Time:
   Waktu tumbuh secara linear dengan ukuran data
   Jika n meningkat 5x, waktu juga meningkat ~5x

3. O(n log n) - Linearithmic Time:
   Waktu tumbuh lebih lambat dari O(n²) tapi lebih cepat dari O(n)
   Tumbuh linear dengan penambahan logaritmik

4. O(n²) - Quadratic Time:
   Waktu tumbuh sangat cepat (eksponensial dari n)
   Jika n meningkat 5x, waktu meningkat ~25x
   Paling lambat di antara semua algoritma

Kesimpulan:
Pola pertumbuhan konsisten dengan teori Big O. Semakin tinggi kompleksitas,
semakin signifikan peningkatan waktu eksekusi seiring bertambahnya input size.

ASUS@LAPTOP-F1OSDSTJ MINGW64 /d/2025573010033_Prak-Struktur-Data-dan-Algoritma (main)
$
```

Output program menampilkan waktu eksekusi dari setiap fungsi pada berbagai ukuran data, sehingga pola pertumbuhan O(1), O(n), O(n log n), dan O(n^2) dapat terlihat sesuai permintaan praktikum.

BAB IV
HASIL DAN PEMBAHASAN

4.1. Hasil Praktikum

Dari hasil praktikum yang telah penulis lakukan, berikut hasil yang di dapat:

1. Direktori praktikum-5 dan tugas berhasil dibuat,
2. File tugas-1.js dan tugas-2.js berhasil dibuat,
3. File tugas-1.js dan tugas-2.js berhasil dijalankan,
4. Project tugas berhasil di push ke repository GitHub.

4.2. Pembahasan

Praktikum ini menunjukkan bahwa setiap algoritma memiliki tingkat efisiensi yang berbeda-beda tergantung pada cara penyusunan logika program. Melalui pengujian menggunakan JavaScript di Node.js, dapat diamati bahwa algoritma sederhana dengan sedikit proses akan menghasilkan waktu eksekusi yang lebih cepat dibandingkan algoritma dengan banyak perulangan. Hal ini membuktikan bahwa struktur algoritma sangat memengaruhi performa program.

Selain waktu eksekusi, penggunaan memori juga menjadi aspek penting dalam analisis kompleksitas. Program yang menggunakan variabel atau struktur data tambahan dalam jumlah besar akan membutuhkan memori lebih banyak. Oleh sebab itu, seorang programmer perlu mempertimbangkan keseimbangan antara kecepatan proses dan efisiensi memori agar aplikasi dapat berjalan optimal pada berbagai perangkat.

Penerapan notasi Big O dalam praktikum ini membantu memahami pola pertumbuhan algoritma saat ukuran data bertambah. Algoritma dengan kompleksitas $O(1)$ dan $O(n)$ cenderung lebih efisien dibandingkan $O(n^2)$ atau $O(2^n)$. Dengan memahami konsep tersebut, praktikan dapat lebih bijak dalam memilih metode penyelesaian masalah sehingga program yang dibuat lebih efektif, cepat, dan mudah dikembangkan di masa depan.

BAB V
PENUTUP

5.1. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa analisis kompleksitas algoritma merupakan langkah penting dalam pengembangan program untuk mengetahui efisiensi waktu dan memori. Konsep Big O mempermudah pengukuran performa algoritma berdasarkan pertumbuhan input data. Melalui praktikum menggunakan JavaScript dan Node.js, pemahaman terhadap pola kompleksitas algoritma dapat diterapkan secara langsung dalam pembuatan program sederhana.

5.2. Saran

Adapun saran yang dapat diberikan adalah sebagai berikut:

1. Praktikan perlu lebih banyak berlatih menganalisis berbagai jenis algoritma.
2. Sebaiknya dilakukan perbandingan beberapa algoritma untuk kasus yang sama.
3. Gunakan data berukuran besar agar perbedaan performa lebih terlihat.
4. Pelajari implementasi algoritma lain seperti sorting dan searching secara lebih mendalam.

DAFTAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data structures and algorithms in Java. Wiley.

McConnell, S. (2004). Code complete (2nd ed.). Microsoft Press.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Flanagan, D. (2020). JavaScript: The definitive guide (7th ed.). O'Reilly Media.